Expand outline and build all this Executive OS

Phase 5 · 6/10 built
The decision layer: operational and strategic metrics together, not raw tables.
Marketplace and partner performance
No screen yet
Gap
Platform core
Phase 1 · 7/10 built
The shared backbone every product depends on: identity, permissions, workflow, notifications, search, files, audit, analytics and integrations.
Identity and lifecycle graph (prospect → alumnus, staff, parent, partner)
One person record across workspaces. No lifecycle graph view yet.
institution/peoplealumni/home
Partial
Organization graph (campus, college, department, cohort, club, housing, provider)
Authority by domain exists. No browsable org graph.
institution/authority
Partial
Reporting warehouse and operational metrics
No warehouse freshness view.
insights/readinessoperations/home
Partial
Student OS
Phase 1 · 11/12 built
One app for everything a student needs to do at school.
Task center, documents, deadlines, signatures
No e-signature step for students.
student/actionsstudent/calendar
Partial
Admissions OS
Phase 2 · 7/10 built
From first inquiry to enrolled student, then a clean hand-off into Student OS.
Prospect profiles and communication history
No communication timeline.
admissions/applicants
Partial
Deposit checkout and payment plan options
applicant/onboarding
Partial

Transfer credit intake
institution/regops
Partial
Academic and Student Success OS
Phase 4 · 7/10 built
Persistence, retention and coordinated intervention, without exposing protected records to social surfaces.
Advising appointments and case notes
No case-notes screen in the main prototype.
advisor/caseloadstudent/advising
Partial
Referral routing (tutoring, counseling, aid, registrar)
institution/handoffs
Partial
Outcome measurement for retention and completion
insights/readiness
Partial
Campus Life OS
Phase 1 · 6/10 built
Engagement and belonging, measurable without disconnected tools.
Membership applications and rosters
student/clubs
Partial
Polls, forms, elections, volunteer sign-ups
student/opportunitiesinstitution/os
Partial
Peer ambassador and orientation communities
applicant/onboarding
Partial
Co-curricular record and badges
student/profile
Partial
Revenue and Finance OS
Phase 3 · 4/11 built
Every money workflow, from charge creation to cash reconciliation. Submodules: Checkout, Billing, Ledger, Reconciliation, Bursar, Refund.
Product catalog (fees, deposits, services, marketplace)
public/pricinginstitution/entitlements
Partial
Installment plans and autopay
Plans shown; no autopay rules.
student/account
Partial
Scholarships, credits and adjustments
finops/queue
Partial

Gateway orchestration (cards, ACH, wallets, refunds)
company/lifecycle
Partial
Refund operations and payout rails
finops/queue
Partial
Collections and bursar holds
finops/queuestudent/registration
Partial
Revenue, cash, aging and write-off reporting
insights/businessinsights/budget
Partial
Partner and Operations OS
Phase 5 · 5/10 built
Runs the outside organizations Semester depends on. This turns the product into an operating system, not just a student app.
Partner directory and relationship history
company/vendors
Partial
Vendor onboarding and verification
company/vendors
Partial
Program-provider inventory (abroad, internships, employers)
employer/rolescareer/postings
Partial
SLA monitoring
institution/os
Partial
Internal notes and cross-team workflows
institution/handoffs
Partial
Executive OS
Phase 5 · 6/10 built
The decision layer: operational and strategic metrics together, not raw tables.
Revenue, cash, aging and collections
insights/business
Partial
Retention and persistence
insights/readiness
Partial
Support workload and SLA reporting
institution/os
Partial
Internal team OS
Ships with every phase · 8/10 built
The tools the Semester team uses to sell, implement, support and improve the platform across

many campuses.
Implementation project tracking
institution/activation
Partial
Usage analytics and adoption monitoring
company/account
Partial
AI and automation layer
Ships with every phase · 6/9 built
Sits across every OS. It recommends, summarizes and routes. It never makes the final call on sensitive actions.
Deadline reminders and outreach campaigns
student/actions
Partial
Matching and routing (advisor, provider, queue, payment exception)
institution/handoffs
Partial
Risk scoring for persistence and delinquency
Persistence only; no delinquency score.
success/risk
Partial

Architecture audit: Semester Faculty OS vs traditional faculty tools — deployment model, security, and AI cost per user
Computer
Interactive prototype: build a faculty dashboard for Semester Faculty OS with live research synthesis and grading queues
Computer
Draft the grading workflow demo in detail
Turn this into a technical architecture spec
Show the policy-controlled gateway code
Sources
129

Built a local interactive Semester completion workbench and a companion gap-matrix CSV covering the capabilities you listed. The workbench includes a filterable cross-domain backlog, Faculty OS modes, a grading-workflow simulation, an executive decision-brief builder, and an editable AI-usage cost calculator.

The governing principle is **decision-first**: every screen should connect evidence to an accountable decision and a confirmed outcome—not merely display another table.

These artifacts are local demonstrations, not a deployed application. They do not retrieve live research, call models, process payments, obtain legally effective signatures, authenticate approvals, or change grades. I also could not retrieve the linked prototype pages, so your “6/10 built,” “7/10 built,” and similar counts remain user-reported coverage indicators—not verified production completion.

1. What the built workbench contains

Component	Functions available locally	What remains for production
Cross-domain gap matrix	Filter by domain, search, mark local review, export JSON	Authenticated ownership, backend persistence, evidence links, permissions
Faculty executive assistant mode	Illustrates scheduling boundaries and approval expectations	Calendar integration, conflict checking, exact approvals, receipts
Research partner mode	Explains source and restricted-data boundaries	Authorized retrieval, citation verification, project isolation
AI TA mode	Illustrates course-policy and answer-key separation	Policy enforcement, approved retrieval, evaluation and escalation
Grading demonstration	Advances through scoring, review, release, partial passback, reconciliation	Isolated runners, authoritative grade integration, real audit evidence
Executive decision brief	Enter question, evidence, alternatives, owner and follow-up; export draft	Verified metrics, authenticated decisions, workflow tracking
AI cost calculator	Calculates model usage from entered volumes and prices	Metering, budget reservations, retries, infrastructure and support allocation

All local review marks and simulated workflow state reset on reload. Exported drafts are planning records—not authenticated approval evidence.

2. Executive OS: the decision layer

Separate two executive contexts:

- Institution Executive OS: academic outcomes, student services, institutional finance, risk, initiatives, and authorized partner performance.
- Semester Company Executive OS: revenue, cash, delivery, customer success, reliability, company obligations, workforce, and ecosystem performance.

They can share components and workflow infrastructure, but must not share unrestricted data access. Your role specification explicitly requires scoped access and does not grant "everything" merely because someone is an administrator or executive. [1]

Proposed Executive OS screens

Screen	Main question	Required functions
Executive Overview	What requires a decision now?	Exceptions, trends, accountable owners, evidence freshness
Decision Inbox	Which decisions are waiting on me?	Exact proposal, alternatives, dependencies, authority, approval expiry
Strategic Initiatives	Are approved initiatives delivering?	Milestones, budget, risks, outcomes, corrective actions
Finance Decision Brief	What explains cash and receivables?	Separate revenue, receivables, settled cash, aging, disputes

Screen	Main question	Required functions
Student Outcomes	Where are persistence and completion changing?	Cohort definitions, exclusions, uncertainty, authorized drill-down
Service Performance	Where is institutional service failing?	Backlog, response/resolution clocks, coverage, handoff failures
Marketplace and Partner Performance	Which relationships should be retained, improved, or retired?	Installation health, service quality, obligations, financial reconciliation
Risk and Trust	Which risks require intervention?	Severity, evidence, mitigation owner, accepted exceptions
Data Confidence	Which metrics are safe to use?	Lineage, load status, freshness, quality, provisional status
Decision History	What was decided, and did it work?	Evidence snapshot, rationale, execution, follow-up, outcome review

These are proposed screen specifications. They should extend the current executive, insights, and operations views rather than create a second reporting system.

Executive metric contract

Every metric should expose:

```
Definition and version
Population / cohort
Reporting period
Source systems
Last complete load
Data watermark
Exclusions and missing data
Calculation owner
Quality status
Publication approval
Permitted drill-down
```

“Updated today” is insufficient if the warehouse only loaded half the expected records.

Marketplace and partner scorecard

This missing screen should combine distinct dimensions rather than produce one opaque score:

- Service: uptime, incidents, unresolved cases, agreed SLA.
- Adoption: authorized installations and completed workflows.
- Quality: verified outcomes, feedback, corrections.
- Trust: access scope, review expiry, data obligations.
- Commercial: invoices, disputes, settlement, approved revenue allocation.
- Decision: renew, remediate, restrict, suspend, or retire.

A relationship can be commercially valuable while having an unresolved security issue. The interface should show that conflict rather than averaging it away.

Executive workflow

```
Metric or exception
  → verify evidence and scope
  → prepare decision brief
  → compare alternatives
  → obtain domain-authorized decision
  → create accountable work
  → execute controlled changes
  → reconcile
  → review outcome
```

AI may prepare the brief and explain uncertainty. It does not independently authorize institutional restrictions, financial actions, disciplinary outcomes, or official academic changes.[2] [1]

3. Platform core: complete the backbone first

Identity and lifecycle graph

“One person across workspaces” should mean a stable identity with separately governed memberships—not one unrestricted merged profile.

Proposed graph objects:

- Person and account.
- Institutional memberships.
- Role grants.
- Course and project relationships.
- Guardian grants.
- Partner affiliations.
- Lifecycle events.
- Source identity mappings.
- Access expiry and revocation.

Example:

```
Person
 ├─ Applicant at Institution A
 ├─ Student at Institution A
 ├─ TA in Course X
 ├─ Employee in Unit Y
 └─ Alumnus after completion
```

These relationships can overlap. Do not replace them with a single mutable `person.role`.

The existing role specification already treats lifecycle transitions, scoped relationships, and access sunset as shared platform responsibilities. [1]

Required screen functions:

1. Browse only permitted relationships.
2. Show effective dates and authoritative source.
3. Identify duplicate or conflicting identities.
4. Propose merge/split resolution with review.
5. Preview the access impact of a transition.
6. Apply approved changes.
7. Reconcile sessions, grants, indexes, integrations, and notifications.

Organization graph

Use typed, effective-dated edges:

Relationship	Meaning
`part_of`	Department belongs to a college
`operates`	Office operates a service
`member_of`	Person belongs to a cohort or organization
`teaches_in`	Faculty assignment
`provides_for`	External organization supplies a permitted service
`authorized_for`	Explicit domain authority

A club, housing assignment, or provider relationship must not imply access to academic or financial records.

Warehouse freshness screen

Show:

- Last successful and last complete load.
- Source event watermark.
- Expected versus received partitions.
- Failed transformations.
- Duplicate and missing-record checks.
- Reconciliation exceptions.
- Metrics affected by the failure.

- Operating owner.
- Replay and correction status.

Suppress or label provisional comparisons when data is incomplete. Do not generate confident executive recommendations from silently stale aggregates.

Preserve the existing permission model

A significant project finding: the reconciliation document says the live repository already has a capability-based model and warns that reference handoff migrations would introduce parallel schemas and permission predicates. It identifies `private.has_capability(...)` and scoped `role_grants` as the existing foundation.[3]

Therefore:

- Extend existing capabilities and canonical records.
- Do not apply reference migrations as if the database were empty.
- Do not introduce another `can()` engine for these new screens.
- Recheck the dated reconciliation findings against the current repository before implementing.

4. Complete the remaining domain workflows

The table below expands your pasted partial features into operational chains. These are proposed requirements, not newly verified implementation status.

Domain	Complete workflow	Important release boundary
Student signatures	Document version → review → applicable consent → identity verification → signature → receipt → validation	Legal sufficiency requires qualified review; a checkbox is not automatically a signature system
Admissions communication	Authorized prospect → consent/preferences → message draft → review → send → delivery/reply → timeline	Applicant communications stay purpose-scoped
Admission deposit	Quote → payer authorization → checkout → pending/settled status → ledger reconciliation → onboarding update	Payment initiation does not equal settlement
Transfer credit	Evidence intake → course identification → equivalency review → approval → record update → degree-audit refresh	Intake is not awarded credit
Advising notes	Case purpose → permitted context → note classification → scoped collaboration → closure/reassignment	Counseling and restricted records remain segregated
Referrals	Request → minimum disclosure → receiving-office acknowledgment → appointment/service → follow-up → closure	Routing does not equal service delivery
Retention outcomes	Cohort definition → complete data → calculation → review → publication → intervention evaluation	Descriptive association is not causal proof

Domain	Complete workflow	Important release boundary
Club membership	Application → eligibility → approval → roster → renewal/expiry → archive	Membership does not reveal unrelated records
Polls/forms/elections	Publish governed form → collect → validate → aggregate/tally → review → results	Ballot secrecy requires a separate design
Orientation	Opt-in → approved cohort/ambassador assignment → activities → escalation → access sunset	Avoid unnecessary applicant-data disclosure
Co-curricular badges	Criteria → evidence → authorized issuer review → issue → student sharing → correction/revocation	Self-entered activity is not verified achievement
Partner onboarding	Purpose → organization verification → agreement → data scope → approval → provisioning → review	Verification is purpose-specific
Partner SLA	Contract calendar → measured events → breach review → remediation → customer notice → closure	Define clock pauses and responsibility
Implementation	Scope → RACI → configuration → migration → training → launch gate → rollout → adoption	“Configured” does not mean authorized
Outreach automation	Purpose → audience → preferences → preview → approval → delivery → outcome	No sensitive details in previews or messages

Finance needs six distinct components

Do not collapse everything into a checkout page.

1. Catalog: approved, effective-dated fees, services, and prices.
2. Billing: charge calculation, statements, adjustments.
3. Ledger: balanced entries and immutable correction history.
4. Payments: provider authorization, processing, settlement.
5. Reconciliation: match provider, bank, ledger, and account state.
6. Operations: disputes, refunds, collections, holds, close.

Autopay additionally needs an explicit authorization/mandate, schedule, revocation, retry policy, notices, and settlement tracking.

Scholarship award decisions, account credits, refunds, and write-offs must remain separate workflows with distinct authority.

For delinquency, begin with factual overdue status and policy-based human review—not an opaque AI score that automatically imposes holds. The project prohibits algorithmic financial decisions and punitive action from workload/risk signals. [2]

5. Faculty OS architecture versus fragmented tools

This comparison is a proposed architecture comparison, not a claim that every existing faculty tool lacks these functions.

Dimension	Fragmented deployment pattern	Target Semester design
Identity	Separate logins and integrations	Shared identity with scoped memberships
Work coordination	Calendar, email, LMS, documents reconciled manually	Canonical work items and Action Center
Source authority	Status spread across tools	Explicit source and authoritative boundary
Research	Separate libraries, notes, analysis, manuscripts	Project-scoped evidence and reproducibility workspace
AI	Separate assistants with varying context	One gateway with mode-specific sources and tools
Grading	Separate scoring, review, release, passback	Versioned scoring and controlled release with reconciliation
Administration	Repeated forms and follow-up	Source-grounded preparation and approved execution
Governance	Controls vary by product	Shared policy, audit, retention, and revocation contracts
Cost	Multiple licenses plus integration effort	Model, infrastructure, integration, and support costs tracked together

The shared-control direction is consistent with Semester's role and intelligence specifications. It still requires implementation and evaluation evidence. [4] [1]

AI cost per faculty user

Use a transparent model:

$$
[
C_{model}
\sum_m
\frac{N_m(I_mP_{i,m}+O_mP_{o,m})}{1{,}000{,}000}
]
$$

Where:

- N_m: monthly runs for mode/model m.
- I_m, O_m: average input/output tokens per run.
- $P_{i,m}$, $P_{o,m}$: provider prices per million tokens.

Then add:

$$
\begin{aligned}
&[\\
&C_{total}
\end{aligned}
$$

$$
\begin{aligned}
&C_{model}\\
&+\\
&C_{retrieval}\\
&+\\
&C_{compute}\\
&+\\
&C_{storage}\\
&+\\
&C_{evaluation}\\
&+\\
&C_{support}\\
&]
\end{aligned}
$$

Use measured workloads and verified provider rates before publishing a price. The workbench calculator intentionally starts provider prices at zero for user entry; that is not a claim that AI is free.

6. Grading workflow demo in detail

The generated workbench demonstrates this chain using synthetic state only:

```
Immutable submission
  → versioned deterministic tests
  → provisional score
  → human review
  → exact release approval
  → partial passback
  → reconciliation
  → correction history
```

Production specification

State	Required behavior
Received	Record submission version, timestamp, receipt, applicable assignment rules
Validated	Check format and submission eligibility without losing the original
Scoring	Run instructor-approved tests in an isolated, resource-limited environment
Provisional	Attach rubric/test version and supporting evidence
Review	Faculty/authorized TA accepts, changes, or escalates
Approved	Bind exact students, submission versions, scores, feedback, and policy version
Submitted	Execute authorized passback once

State	Required behavior
Confirmed	Record source-system acceptance
Reconciling	Update permitted grade views and identify differences
Exception	Keep rejected/pending records visible and owned
Corrected	Preserve earlier value and reason; follow correction authority

Failure demonstration

Suppose a synthetic release batch contains three results:

- Two accepted.
- One rejected because its authoritative roster mapping changed.

The interface must show “2 confirmed, 1 exception”—not “release successful.”

Retry only the permitted unresolved command. If its target, score, or material evidence changes, obtain renewed approval.

AI-assisted grading restriction

The current project documents prohibit sending student records to AI providers. The research synthesis panel and AI feedback demonstrations therefore must use approved non-student-specific material or synthetic examples unless a separately reviewed processing architecture is authorized. [5] [4]

Deterministic autoscoring, model-generated feedback, and official grade release remain three separate capabilities.

7. Policy-controlled gateway code

Below is a proposed orchestration contract—not inspected repository code, a new endpoint, or a production-ready implementation. Its adapters must bind to the existing permission model and policy journal rather than introduce parallel authorization.

```
type Mode = "executive" | "research" | "teaching";

type Scope = {
  kind: "school";
  id: string;
};

type ResourceRef = {
  kind: string;
  id: string;
};

type AIRequest = {
  mode: Mode;
  resource: ResourceRef;
```

```
  prompt: string;
  maxOutputTokens: number;
};

type Policy = {
  version: string;
  aiEnabled: boolean;
  externalProvidersAllowed: boolean;
  allowedProviderIds: string[];
  maxOutputTokens: number;
};

type Source = {
  id: string;
  version: string;
  content: string;
  classification: string;
};

interface GatewayDependencies {
  authenticate(sessionToken: string): Promise<{ userId: string }>;

  resolveAuthorizedScope(
    userId: string,
    resource: ResourceRef
  ): Promise<Scope>;

  authorizeRequest(
    userId: string,
    scope: Scope,
    request: AIRequest
  ): Promise<void>;

  effectivePolicy(
    scope: Scope,
    request: AIRequest
  ): Promise<Policy>;

  checkKillSwitch(scope: Scope): Promise<void>;

  retrieveAuthorizedSources(
    userId: string,
    scope: Scope,
    request: AIRequest
  ): Promise<Source[]>;

  validateOutboundPayload(
    policy: Policy,
    prompt: string,
    sources: Source[]
  ): Promise<void>;

  chooseApprovedProvider(policy: Policy): Promise<string>;

  reserveBudget(
    scope: Scope,
```

```
    request: AIRequest,
    providerId: string
  ): Promise<{ reservationId: string }>;

  revalidateBeforeInvocation(
    userId: string,
    scope: Scope,
    request: AIRequest,
    policyVersion: string,
    sources: Source[]
  ): Promise<void>;

  invokeWithoutTools(input: {
    providerId: string;
    prompt: string;
    sources: Source[];
    maxOutputTokens: number;
  }): Promise<{ text: string; usage: unknown }>;

  validateAndStoreDraft(input: {
    userId: string;
    scope: Scope;
    request: AIRequest;
    policyVersion: string;
    sources: Source[];
    text: string;
  }): Promise<{ draftId: string }>;

  recordRunMetadata(input: {
    userId: string;
    scope: Scope;
    draftId: string;
    policyVersion: string;
    sourceVersions: string[];
    providerId: string;
    usage: unknown;
  }): Promise<void>;

  finalizeBudget(
    reservationId: string,
    usage: unknown
  ): Promise<void>;

  settleFailedRun(reservationId: string): Promise<void>;
}

export async function prepareFacultyDraft(
  deps: GatewayDependencies,
  sessionToken: string,
  request: AIRequest
) {
  const actor = await deps.authenticate(sessionToken);

  const scope = await deps.resolveAuthorizedScope(
    actor.userId,
    request.resource
```

```
  );

  await deps.authorizeRequest(actor.userId, scope, request);
  await deps.checkKillSwitch(scope);

  const policy = await deps.effectivePolicy(scope, request);

  if (!policy.aiEnabled || !policy.externalProvidersAllowed) {
    throw new Error("External AI processing is not permitted.");
  }

  if (
    !Number.isInteger(request.maxOutputTokens) ||
    request.maxOutputTokens <= 0 ||
    request.maxOutputTokens > policy.maxOutputTokens
  ) {
    throw new Error("Invalid output-token limit.");
  }

  const sources = await deps.retrieveAuthorizedSources(
    actor.userId,
    scope,
    request
  );

  // Must inspect prompt and retrieved content.
  // A classification label alone is not a sufficient egress check.
  await deps.validateOutboundPayload(
    policy,
    request.prompt,
    sources
  );

  const providerId = await deps.chooseApprovedProvider(policy);
  const budget = await deps.reserveBudget(
    scope,
    request,
    providerId
  );

  try {
    await deps.revalidateBeforeInvocation(
      actor.userId,
      scope,
      request,
      policy.version,
      sources
    );

    const result = await deps.invokeWithoutTools({
      providerId,
      prompt: request.prompt,
      sources,
      maxOutputTokens: request.maxOutputTokens
    });
```

```
    await deps.finalizeBudget(
      budget.reservationId,
      result.usage
    );

    const draft = await deps.validateAndStoreDraft({
      userId: actor.userId,
      scope,
      request,
      policyVersion: policy.version,
      sources,
      text: result.text
    });

    await deps.recordRunMetadata({
      userId: actor.userId,
      scope,
      draftId: draft.draftId,
      policyVersion: policy.version,
      sourceVersions: sources.map(s => `${s.id}:${s.version}`),
      providerId,
      usage: result.usage
    });

    return {
      status: "draft_only",
      draftId: draft.draftId,
      policyVersion: policy.version
    };
  } catch (error) {
    // Must reconcile actual provider usage on uncertain failures.
    await deps.settleFailedRun(budget.reservationId);
    throw error;
  }
}
```

What this code deliberately does not do

- It does not accept a trusted tenant identifier from the browser.
- It does not execute model-proposed tools.
- It does not publish content, send email, write calendars, or release grades.
- It does not rely on the model to decide its own access.
- It does not claim that classifications alone prevent sensitive-data leakage.
- It does not log full prompts or source payloads as routine metadata.

Approved writes belong in the existing controlled-command path, with fresh authorization, exact approval, duplicate protection, audit, and reconciliation. The project reconciliation document specifically warns against duplicating the current capability and approval systems. [3]

8. Build order and activation gates

Order	Work	Accountable owner	Exit evidence
1	Reconcile existing models, screens, capabilities, and migrations	Platform + product	No duplicate permissions or canonical records
2	Complete identity/org graph and data-confidence contracts	Identity + data engineering	Scope, lifecycle, lineage, revocation tests
3	Harden gateway and shared commands	Security + AI/platform engineering	Denial, egress, policy, budget, audit tests
4	Complete faculty and executive demonstrations	Faculty/executive product owners	End-to-end synthetic acceptance
5	Connect authorized read-only sources	Integration + institution owners	Mapping, freshness, provenance, isolation
6	Enable assist-mode cohorts	Institution + release owner	Training, support, evaluation, monitoring
7	Add bounded publication/scheduling actions	Domain owners	Exact approval and receipt/reconciliation tests
8	Activate financial/signature/grade workflows	Appropriate institutional and professional reviewers	Domain-specific authorization and operating evidence
9	Transfer official authority where approved	Record steward + IT + implementation	Dual run, cutover, reconciliation, recovery acceptance

Your intelligence evidence still needs reconciliation: one document claims provider exclusion is red-team tested, while another says the red-team suite has not run. Resolve that against a dated test report for the actual gateway version before activating sensitive processing. [4] [5]

The generated workbench and CSV now provide a concrete design-review surface for this work. They prepare implementation; they do not establish that the listed phase fractions are accurate, that live systems are connected, or that any module is authorized for production.

⁂

1. The Main Semester design system (2) copy/docs/system/ROLE_LIFECYCLES.md
2. The Main Semester design system (2) copy/docs/system/algorithms/SEMESTER_ALGORITHM_GOVERNANCE.md
3. The Main Semester design system (2) copy/docs/system/ROLE_SYSTEM_RECONCILIATION.md
4. The Main Semester design system (2) copy/docs/platform/AI_INTELLIGENCE_PLATFORM.md
5. The Main Semester design system (2) copy/docs/system/algorithms/SEMESTER_AI_ROUTING.md